# User document
# YALTA

David Avanessoff, Catherine Bonnet, Hugo Cavalera
André Fioravanti and Le Ha Vy Nguyen

28 September 2015 - v 1.0.1

## Contents

# 1 The class of delay systems YALTA can handle

YALTA is a MATLAB Toolbox dedicated to the stability analysis of (fractional) delay systems given by their transfer function.

YALTA considers the class of systems with transfer function of the type :

$$G(s) = \frac{t(s) + \sum_{\kappa=1}^{N'} t_\kappa(s) e^{-\kappa s\tau}}{p(s) + \sum_{k=1}^{N} q_k(s) e^{-ks\tau}} = \frac{n(s)}{d(s)} \tag{1}$$

where

$\tau > 0$, is the nominal delay

$t$, $p$, $q_k$ for all $k \in \mathbb{N}_N$, and $t_\kappa$ for all $\kappa \in \mathbb{N}_{N'}$, are real polynomials in $s^\alpha$ for $0 < \alpha \leq 1$ which satisfy

$\deg p \geq \deg t$,

$\deg p \geq \deg t_\kappa$ for all $\kappa \in \mathbb{N}_{N'}$,

$\deg p \geq \deg q_k$ for all $k \in \mathbb{N}_N$.

Here, the degree is interpreted as the degree in $s^\alpha$.

If $\deg p = \deg q_k$ for at least one $k \in \mathbb{N}_N$, $G$ defines a *neutral* time-delay system, otherwise it will define a *retarded* system [2].

We write $z = e^{-s\tau}$ and suppose that for each $k$

$$\frac{q_k(s)}{p(s)} = \alpha_k + \mathcal{O}(s^{-\alpha}) \qquad \text{as} \quad |s| \to \infty, \tag{2}$$

The coefficient of the highest degree term of $p(s) + \sum_{k=1}^{N} q_k(s) e^{-ks\tau}$ can then be written as a multiple of the following polynomial in $z$

$$\tilde{c}_d(z) = 1 + \sum_{i=1}^{N} \alpha_i z^i. \tag{3}$$

**Hypothesis (H1):** The roots of $\tilde{c}_d$ are of multiplicity one.

**Hypothesis (H2):** The polynomials $p(s)$ and $q_k(s)$ satisfy

$$p(0) + \sum_{k=1}^{N} q_k(0) \neq 0.$$

**Hypothesis (H3):** In order to avoid the possibility of an infinite number of zero cancellations between the numerator and denominator of $G$, we suppose that the numerator of $G$ satisfies either

a) $\deg t(s) > \deg t_k(s)$; or

b) $\deg t_k(s) = \deg t(s)$ for at least one $k$ and the polynomial $\tilde{c}_n$ defined as in (3) relatively to the quasi-polynomial $n(s)$ has no root of modulus less than or equal to one, and no common root of modulus strictly greater than one with $\tilde{c}_d$.

# 2 The functionalities of YALTA

The questions YALTA can answer are :

- In the case of neutral systems
  - Find the position of asymptotic axes.
  - If the imaginary axis is an asymptotic axis, find if the asymptotic poles of the chain are left or right the imaginary axis.
- In the case of retarded systems or of neutral systems with asymptotic axes in $\{\mathrm{Re}\, s < 0\}$, find:
  - For a given $\tau$, the number and the position of unstable poles.
  - For which values of $\tau$ the system is stable;
  - For a set of values of the delay, the position of unstable poles (root locus).
  - The coprime factors $(N, D)$ of $G$ as well as an approximation $(N_n, D_n)$ in $H_\infty$-norm

YALTA is based on [3, 4, 5, 6, 1, 8, 7].

# 3 delayFrequencyAnalysisMin

Let us consider the characteristic equation :

$$C(s,\tau) = \sum_{k=0}^{N} q_k(s) e^{-ks\tau} \tag{4}$$

## 3.1 Syntax

```
T = delayFrequencyAnalysisMin(iPolyMatrix,iDelayVect,iAlpha,iTau,...
   ibPlotOption,iDeltaTau,tminsw,tmaxsw)
T = delayFrequencyAnalysisMin(iPolyMatrix,iDelayVect,iAlpha,iTau,...
   ibPlotOption,iDeltaTau)
T = delayFrequencyAnalysisMin(iPolyMatrix,iDelayVect,iAlpha,iTau,...
   ibPlotOption)
T = delayFrequencyAnalysisMin(iPolyMatrix,iDelayVect,iAlpha,iTau)
```

## 3.2 Inputs

- iPolyMatrix: polynomials $(q_k)_{k=0..N}$ in the following matrix form

$$\begin{bmatrix} q_{0,n} & q_{0,n-1} & \cdots & q_{0,0} \\ \cdots & & & \\ q_{N,n} & q_{N,n-1} & \cdots & q_{N,0} \end{bmatrix} \tag{5}$$

  where $q_k(s) = \sum_{j=0}^{n} q_{k,j}(s^\alpha)^j$.

Note that with the following input, you will not have to explicitly write the null polynomials $q_k$.

- iDelayVect: a vector of values of k for which $q_k$ is not null of length the number of rows of the above matrix minus one ($q_0$ is assumed non zero).
- iAlpha: a real between 0 and 1 giving the power $\alpha$ of $s$ for fractionary polynomials in $s^\alpha$.
- iTau: the nominal value of the delay, denoted $\tau$.
- ibPlotOption: a boolean variable determining if a graph of root loci is traced (1) or not (0), default value is 1.
- iDeltaTau: precision of integration procedure for computing root locus, default value is $10^{-4}$.
- tminsw: the minimum delay of stability window, default value is 0.
- tmaxsw: the maximum delay of stability window, default value is $\tau$.

**Example 3.1.**

```
>> q = [1,3,2,-1;
    0,3,-4,2;
    0.5,0.7,1.2,-0.8;
    0.5,1.34,-0.7,1.9;
    0,0,3.4,-1.6]
q =
    1.0000    3.0000    2.0000   -1.0000
         0    3.0000   -4.0000    2.0000
    0.5000    0.7000    1.2000   -0.8000
    0.5000    1.3400   -0.7000    1.9000
         0         0    3.4000   -1.6000
>> DelayVector = [1,2,3,6]
DelayVector =
     1     2     3     6
>> Example1 = delayFrequencyAnalysisMin(q,DelayVector,1,2,1)
```

## 3.3 Outputs

Before saying a word on the outputs, let us check the result of the last command as an overview.

```
>> Example1 = delayFrequencyAnalysisMin(q,DelayVector,1,2,1)
Example1 =
      AsympStability: 'There is(are) 4 unstable pole(s) in right half plane'
                Type: 'Neutral'
         RootsNoDelay: [-3.7865 -0.1167 + 0.2289i -0.1167 - 0.2289i]
           RootsChain: [-0.0413 -0.2640]
        CrossingTable: [7x4 double]
       ImaginaryRoots: [4x3 double]
    StabilityWindows: [2x3 double]
     NbUnstablePoles: [2x4 double]
       UnstablePoles: [0.2360 - 0.2325i 0.2360 + 0.2325i 0.0762 - 1.7797i 0.0762 + 1.779
               Error: [1.0000e-10 1.0000e-10 1.0000e-10 1.0000e-10]
            RootLoci: {8x1 cell}
```

As you can see, the output is a compact structure where some parts have to be asked to be displayed.

```
>> Example1.RootsNoDelay'
ans =
  -3.7865
  -0.1167 - 0.2289i
  -0.1167 + 0.2289i
>> Example1.RootsChain'
ans =
   -0.0413
   -0.2640
>> Example1.CrossingTable
ans =
    0.3868   10.2144    0.6151    1.0000
    0.9351    1.8698    3.3603    1.0000
    2.5057    3.6611    1.7162    1.0000
    3.0461    5.0862    1.2353   -1.0000
    3.8342   16.1150    0.3899    1.0000
    4.9367    5.7489    1.0929    1.0000
   29.1116   29.5088    0.2129   -1.0000
>> Example1.ImaginaryRoots
ans =
    0.3868    0.6151    1.0000
    0.3868   -0.6151    1.0000
    0.9351    3.3603    1.0000
    0.9351   -3.3603    1.0000
>> Example1.StabilityWindows
ans =
```

```
         0    0.3868    2.0000
    1.0000         0         0
>> Example1.NbUnstablePoles
ans =
         0    0.3868    0.9351    2.0000
         0    2.0000    4.0000    4.0000
>> Example1.UnstablePoles'
ans =
   0.2360 + 0.2325i
   0.2360 - 0.2325i
   0.0762 + 1.7797i
   0.0762 - 1.7797i
>> Example1.Error'
ans =
   1.0e-10 *
    1.0000
    1.0000
    1.0000
    1.0000
```

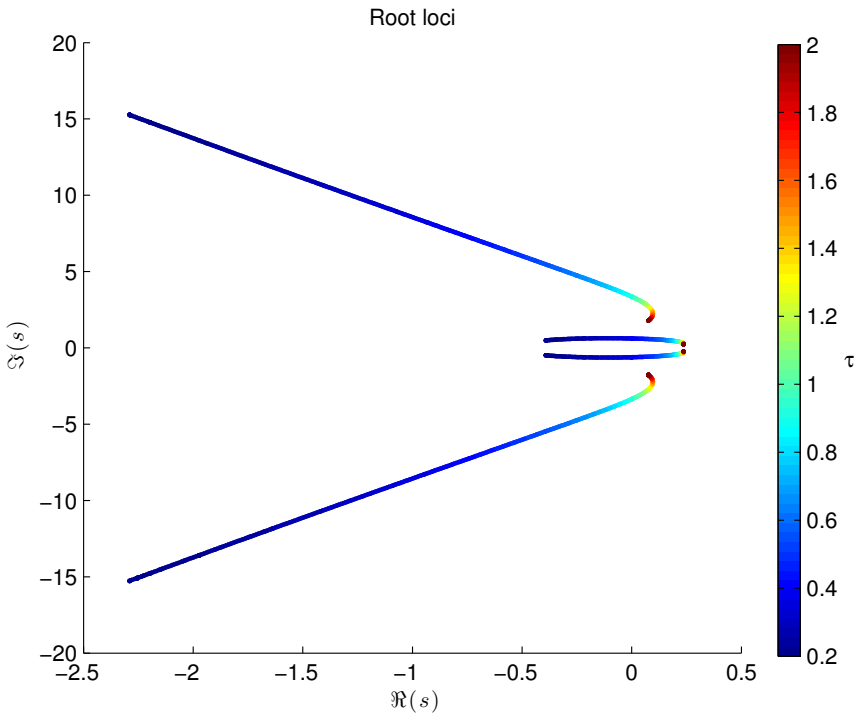


Figure 1: Root loci of the system in Example 3.1

- As we have just seen, first output is a structure giving the different expected informations on the equation. We will have a look on this more in details.

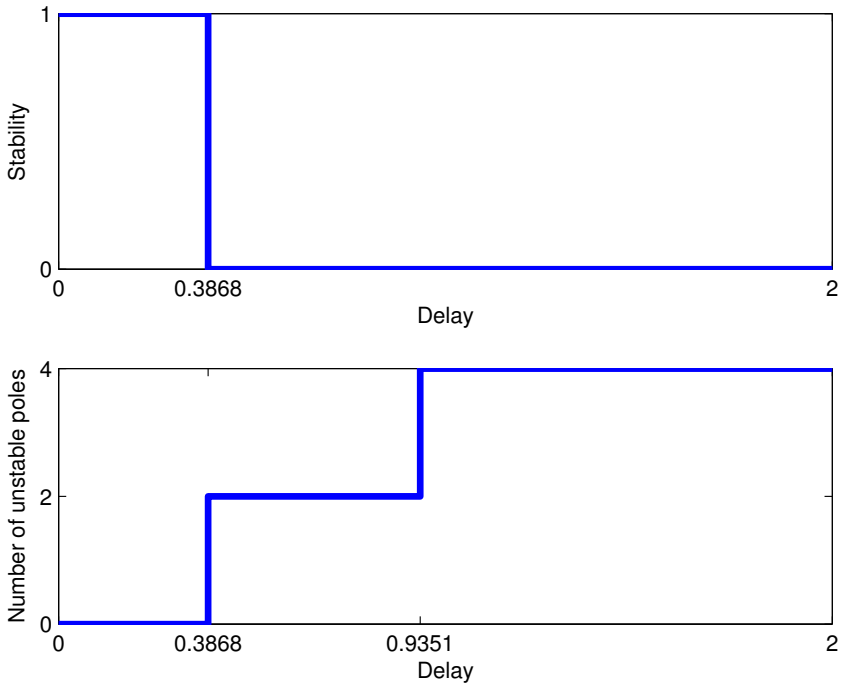


Figure 2: Stability window and number of unstable poles of the system in Example 3.1 in the delay interval [0,2]

- A colored graph (from deep blue for small delays to deep red for high delays, depending of the maximum delay entered) giving the evolution of the roots in function of the delay. Indeed, this graph is in 3-D and the delay corresponding to each point of the root loci is given by the z coordinate. A simple way to view the delay is to use *Data cursor* available in Matlab figure windows.
- Two graphs showing the stability and the number of unstable poles in the specified delay interval.

### 3.3.1 AsympStability

The AsympStability is either:

- 'Infinite number of unstable poles'
- 'Impossible to conclude' in the case where YALTA cannot determine if a chain of poles clustering the imaginary axis is left or right the axis (see [3])
- 'There is no unstable pole'
- 'There are 'd' unstable poles'

### 3.3.2 Type

This output gives the type of the system [2]: retarded, neutral or advanced (in case the user was wrong in defining his/her system).

### 3.3.3 RootsNoDelay

An array of roots of the non delayed system.

### 3.3.4 RootsChain

For neutral systems: An array of abscissa of the asymptotes of the roots chains.

For retarded systems: 'Roots chains only computed for neutral systems'

### 3.3.5 CrossingTable

This array describes the points of crossing of the imaginary axis. The first column gives the first value of the delay for which a specific zero crosses the axis. The third column gives the frequency of crossing of a zero. The second column is two times pi over frequency. The fourth column gives the crossing direction, $-1$ is from right to left and 1 from left to right.

### 3.3.6 StabilityWindows

The first row gives the values of delay where the stability changes, i.e. from stability to instability or vice versa, including the minimum and maximum delays. The second row indicates the stability of delay intervals. The $i$-th entry of the second row corresponds to the delay interval from $i$-th to $(i+1)$-th entries of the first row.

### 3.3.7 NbUnstablePoles

This array is similar to *StabilityWindows* array except that the first row contains the delays where the number of unstable poles change and the second row gives the number of unstable poles in delay intervals.

### 3.3.8 ImaginaryRoots

The set of all the imaginary roots for a delay between zero and the delay $\tau$ given in input. The first column is the delay value, the second column the position on the imaginary axis and the third column gives the direction as in the CrossingTable output.

### 3.3.9 UnstablePoles

The set of all the unstable poles at the delay value given in input (nominal delay).

### 3.3.10 Error

The error of evaluation of unstable poles.

### 3.3.11 RootLoci

This cell array contains branches of the displayed root loci (root loci starting from unstable poles of delay free system and taking into account those crossing the imaginary axis from left to right). Each matrix of the cell represents a branch. It has three rows containing the real, imaginary parts and the corresponding delay of points of each branch.

# 4 thread_Analysis

In order to obtain better performances, one can use the function thread_Analysis to get only some of the main outpus of the function *delayFrequencyAnalysisMin*.

## 4.1 Syntax

```
T = thread_Analysis(iPolyMatrix, iDelayVect, iAlpha, iTau)
```

## 4.2 Inputs

Please refer to subsection 3.2 for more details on inputs.

## 4.3 Outputs

- AsympStability
- Type
- RootsNoDelay
- RootsChain
- CrossingTable
- ImaginaryRoots

Refer to subsubsections 3.3.1, 3.3.2, 3.3.3, 3.3.4, 3.3.5 and 3.3.8 for more details.

# 5 thread_StabilityWindows

This function can be used if one only wants to have a graph of the stability windows for a bounded set of values of the delay.

## 5.1 Syntax

```
T = thread_StabilityWindows(iPolyMatrix, iDelayVect, iAlpha, iTau, ...
                            tminsw, tmaxsw, plot)
```

## 5.2 Inputs

Please refer to subsection 3.2 for more details on inputs.

### 5.3 Outputs

- Type
- RootsNoDelay
- RootsChain
- CrossingTable
- StabilityWindows
- NbUnstablePoles
- Two graphs for the stability windows and the numbers of unstable poles

Refer to subsubsections 3.3.2, 3.3.3, 3.3.4, 3.3.5, 3.3.6 and 3.3.7 for more details on the outputs.
As an example, see Figure 2 for the plot of stability windows and number of unstable poles.

## 6 thread_RootLoci

This function computes the Root Locus of the system, starting from unstable poles of the delay free system and taking into account those crossing the imaginary axis from left to right.

### 6.1 Syntax

```
T = thread_RootLoci(iPolyMatrix, iDelayVect, iAlpha, iTau, ibPlotOption, iDeltaTau)
```

### 6.2 Inputs

Please refer to subsection 3.2 for more details on inputs.

### 6.3 Outputs

- RootsNoDelay
- CrossingTable
- ImaginaryRoots
- UnstablePoles
- Error
- RootLoci

Refer to subsubsections 3.3.3, 3.3.5, 3.3.8, 3.3.9, 3.3.10 and 3.3.11 for more details.

# 7 Pade2 approximation (for non fractional systems)

**This function can only be used if user possesses the Control System Matlab Toolbox.**

Consider $\tilde{C}(s) = \frac{d(s)}{(s+1)^\delta}$, with $\delta$ greater or equal to $n+1$ where $n$ is the polynomial degree of $d$ with $d$ given as in (1) (*ie* $n = \deg p$) and $\alpha = 1$.

## 7.1 computePade

This is the main function giving the Pade2 approximation [7, 8] of transfer functions of the type $\tilde{C}(s)$.

There are two modes, the user can specify : either the maximum limit of the difference ($H_\infty$ norm) between $\tilde{C}$ and its approximation or the order of the approximation.

**Note:** To get an approximation of $G(s)$ defined as in (1) and having a finite number of unstable poles, let us write $G(s) = \frac{\frac{n(s)}{(s+1)^{n+1}}}{\frac{d(s)}{(s+1)^{n+1}}} = \frac{N(s)}{D(s)}$. If $N$ and $D$ do not have commun unstable zeros (this can be verified with thread_RootLoci) this factorization $(N, D)$ is indeed a coprime factorization of $G$.

Let $N_k(s)$ and $D_k(s)$ be respectively Pade2 approximants of $N(s)$ and $D(s)$, an approximation of $G(s)$ is given by $G_k(s) = \frac{N_k(s)}{D_k(s)}$.

### 7.1.1 Inputs

(iPolyMatrix, iDegree, iTau, iDelayVector, iModArg, iMode)

- iPolyMatrix: polynomials $(q_k)_{k=0..N}$ in the following matrix form

$$\begin{bmatrix} q_{0,n} & q_{0,n-1} & \cdots & q_{0,0} \\ \cdots & & & \\ q_{N,n} & q_{N,n-1} & \cdots & q_{N,0} \end{bmatrix} \tag{6}$$

  where $q_k(s) = \sum_{j=0}^{n} q_{k,j} s^j$.

Note that with the following input, you will not have to explicitly write the null polynomials $q_k$.

- iDegree: the degree of the denominator of $\tilde{C}$.
- iTau: the nominal value of the delay.
- iDelayVector: a vector of values of k for which $q_k$ is not null of length the number of rows of the above matrix minus one ($q_0$ is assumed non zero).
- iModArg: the argument of the chosen mode, either the order of the approximation or the maximum difference between the norms.
- iMode: either 'ORDER' or 'NORM' string choosing thus the mode.

### 7.1.2 Outputs

The output is a structure of the following form :

- PadeStruct.NumApprox : vector of coefficients of the numerator of the transfert function of the approximation.
- PadeStruct.DenApprox : vector of coefficients of the denominator of the transfert function of the approximation.
- PadeStruct.ErrorNorm : the $H_\infty$-norm of the difference between $\tilde{C}$ and its approximation.
- PadeStruct.PadeOrder : the order of the approximation.
- PadeStruct.Roots : the roots of the approximation
- PadeStruct.RootsError : an array of differences between the computed roots of $\tilde{C}$ (by thread_RootLoci) and the roots of the approximation.

# 8 Performance

Plotting root loci generally takes a lot of time. Fortunately, this task can be realized by parallel computation since root loci do not depend on each other. If you have Parallel Computing Toolbox in your Matlab version and a multi-core processor, you can execute the following command

```
>> matlabpool open 2 % use 2 parallel cores
```

before calling YALTA. To deactivate parallel computation,

```
>> matlabpool close
```

# References


[1] D. Avanessoff, A.R. Fioravanti, and C. Bonnet. Yalta: a matlab toolbox for the $h_\infty$-stability analysis of classical and fractional systems with commensurate delays. In *IFAC Joint Conference, 11th Workshop on Time-Delay Systems*, February 2013.

[2] R. Bellman and K. L. Cooke. *Differential-Difference Equations*. Academic Press, New York, London, 1963.

[3] C. Bonnet, A. R. Fioravanti, and J.R. Partington. Stability of neutral systems with commensurate delays and poles asymptotic to the imaginary axis. *SIAM Journal on Control and Optimization*, 49:498–516, 2011.

[4] A.R. Fioravanti, C. Bonnet, and H. Ozbay. Stability of fractional neutral systems with multiple delays and poles asymptotic to the imaginary axis. In *IEEE Conference on Decision and Control*, Atlanta, USA, December 2010.

[5] A.R. Fioravanti, C. Bonnet, H. Ozbay, and S.-I. Niculescu. Stability windows and unstable poles for linear time-delay systems. In *9th IFAC Workshop on Time-Delay Systems*, Prague, June 2010.

[6] A.R. Fioravanti, C. Bonnet, Hitay Ozbay, and S.-I Niculescu. A numerical method for stability windows and unstable root-locus calculation for linear fractional time-delay systems. *Automatica*, 48(11):2824–2830, November 2012.

[7] P. M. Mäkilä and J. R. Partington. Laguerre and Kautz shift approximations of delay systems. *Internat. J. Control*, 72(10):932–946, 1999.

[8] J.R. Partington. Approximation of unstable infinite-dimensional systems using coprime factors. *Systems and Control Letters*, 16:89–96, 1991.